

Web Technologies II

Chapter One: Introduction to React and Modern JavaScript

1. Introduction to React Library

React, also known as React.js or ReactJS, is a popular front-end JavaScript library developed by Facebook (engineer Jordan Walke). Its primary purpose is to build highly interactive user interfaces (UIs) by focusing on the "View" aspect of the Model-View-Controller (MVC) architecture. Instead of manipulating the browser's Document Object Model (DOM) directly, React creates an in-memory representation called the Virtual DOM, which allows it to efficiently update only the necessary parts of the actual UI when data changes.

2. Setting Up a React Project with Vite

Vite is a modern build tool that offers a faster and more lightweight development experience for React projects. The process for setting up a project is as follows:

- **Prerequisites:** Download and install Node.js from <https://nodejs.org>. After installation, verify it by running `node -v` and `npm -v` in your terminal.
- **Creating a New Project:** Run the command `npm create vite@latest` in your terminal.
- **Configuration:** You will be prompted to:
 1. Provide a project name (e.g., `my-react-app`).
 2. Select `React` as the framework.
 3. Choose either `JavaScript` or `TypeScript` as the variant.
- **Installing Dependencies:** Navigate into your project directory (`cd my-react-app`) and run `npm install` to install all necessary dependencies.
- **Starting the Development Server:** Run `npm run dev`. This will start a local development server, typically accessible at `http://localhost:5173`. The development server provides features like hot module replacement, allowing you to see changes in real-time.

- **Building for Production:** To create an optimized production build, run `npm run build`. The output files will be placed in a `dist/` directory.

3. Project Structure

A typical Vite-powered React project has the following structure:

```
my-react-app/
├── node_modules/      # All project dependencies
├── public/            # Static assets (e.g., images, favicons)
├── src/               # Source code for the application
│   ├── assets/        # Images, fonts, etc. imported into components
│   ├── App.jsx        # The main root component of the application
│   ├── main.jsx       # The entry point that renders the App component into the DOM
│   └── index.css       # Global styles
├── .gitignore         # Specifies files/folders to be ignored by Git
├── index.html         # The main HTML file that loads the React app
├── package.json       # Project metadata and dependencies list
├── vite.config.js     # Configuration file for the Vite build tool
└── README.md          # Project documentation
```

4. Core Concepts of React

- **Virtual DOM:** This is a lightweight, in-memory copy of the real DOM. When a component's state changes, React creates a new Virtual DOM tree, compares it with the previous one (a process called "diffing"), and then efficiently updates only the changed parts in the actual browser DOM ("reconciliation"). This process significantly improves performance.
- **Component-Based Architecture:** The UI is broken down into small, reusable, and independent pieces called components. Components can be

nested within other components, promoting code reusability, maintainability, and scalability. There are two main types:

- **Functional Components:** These are simple JavaScript functions that return JSX (JavaScript XML). They are the modern standard and use Hooks for state and side effects.
- **Class Components:** These are ES6 classes that extend `React.Component` and have a `render()` method that returns JSX. While they can manage state and lifecycle methods, they are less common in modern React development due to the introduction of Hooks.
- **JSX (JavaScript XML):** JSX is a syntax extension that allows you to write HTML-like code directly within JavaScript. It makes the code more readable, expressive, and easier to debug. JSX is not understood by browsers directly; it is transpiled by a compiler like Babel into standard JavaScript `React.createElement()` calls. You can embed any valid JavaScript expression inside JSX using curly braces `{}`.

```
const name = "John";  
const element = <h1>Hello, {name}!</h1>;
```

- **One-Way Data Binding:** Data flows in a single direction, from parent components to child components via **props**. This unidirectional flow makes the application's behavior more predictable and easier to debug.
- **State Management:** **State** is data that a component manages internally and can change over time. When a component's state changes, React automatically re-renders that component and its children. In functional components, state is managed using the `useState` Hook. State allows for dynamic UI updates without needing to reload the page.
- **React Hooks:** Hooks are functions that allow functional components to "hook into" React state and lifecycle features. Common Hooks include:
 - `useState`: For managing local state within a component.
 - `useEffect`: For handling side effects such as fetching data from an API, setting up subscriptions, or manually changing the DOM.
 - `useContext`: For accessing global state without having to pass props down through multiple levels (prop drilling).

5. Modern JavaScript (ES6+) Features Crucial for React

React heavily leverages modern JavaScript syntax. Key features include:

- **Arrow Functions:** Provide a concise syntax for writing functions. They also lexically bind the `this` value, which is important for callbacks and event handlers.

```
// Single parameter (parentheses optional)
const square = x => x * x;
// Multiple parameters (parentheses required)
const add = (a, b) => a + b;
// No parameters (empty parentheses)
const greet = () => "Hello, World!";
// Multi-line body (curly braces and return statement required)
const sumSquares = (a, b) => {
  const sum = a*a + b*b;
  return sum;
};
```

- **Array Methods:** These methods provide powerful ways to manipulate arrays without mutating the original array.
 - `map()`: Creates a new array by applying a function to each element of the original array. This is extremely common for rendering lists in React.

```
const numbers = [1, 2, 3];
const doubled = numbers.map(n => n * 2); // [2, 4, 6]
```

- `filter()`: Creates a new array containing only elements that pass a test.

```
const numbers = [1, 2, 3, 4, 5];
const evenNumbers = numbers.filter(n => n % 2 === 0);
// [2, 4]
```

- `reduce()`: Reduces an array to a single value by accumulating a result.

```
const numbers = [1, 2, 3];
const sum = numbers.reduce((total, num) => total + nu
```

```
m, 0); // 6
```

- **Destructuring:** A convenient way to extract values from arrays or properties from objects into distinct variables.

```
// Array destructuring
const colors = ["red", "green", "blue"];
const [primary, secondary, tertiary] = colors;
console.log(primary); // "red"

// Object destructuring
const person = { name: "Alice", age: 25 };
const { name, age } = person;
console.log(name); // "Alice"
```

- **Spread Operator (...):** Allows an iterable (like an array or object) to be expanded into individual elements or key-value pairs. It's useful for creating copies or combining data.

```
// Array spread (shallow copy)
const fruits = ["apple", "banana"];
const moreFruits = [...fruits, "orange"]; // ["apple",
"banana", "orange"]

// Object spread
const obj = { a: 1, b: 2 };
const objCopy = { ...obj, c: 3 }; // { a: 1, b: 2, c: 3
}
```

- **Rest Operator (...):** Allows a function to accept an indefinite number of arguments as an array or to gather remaining properties in destructuring.

```
// Function rest parameters
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}

// Object rest in destructuring
```

```
const obj = { a: 1, b: 2, c: 3 };
const { a, ...rest } = obj;
console.log(rest); // { b: 2, c: 3 }
```

- **Classes:** Syntactic sugar over JavaScript's prototypal inheritance, providing a structured way to define objects with methods and constructors.

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hi, I'm ${this.name}.`);
  }
}

class Student extends Person {
  constructor(name, grade) {
    super(name);
    this.grade = grade;
  }
  study() {
    console.log(`${this.name} is studying in grade ${this.grade}.`);
  }
}
```

- **Promises and Async/Await:** Essential for handling asynchronous operations (like API calls) without blocking the main thread.
 - **Promises:** An object representing the eventual completion (or failure) of an asynchronous operation. They have states: pending, fulfilled, or rejected. They are handled with `.then()`, `.catch()`, and `.finally()`.
 - **Async/Await:** Syntactic sugar built on top of Promises, allowing you to write asynchronous code that looks and behaves more like synchronous code. An `async` function always returns a Promise, and the

`await` keyword pauses the execution of the function until a Promise is resolved.

```
async function fetchData() {
  try {
    const response = await fetch('<https://api.examp
e.com/data>');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Error:", error);
  }
}
```

- **HTTP Requests:** In the frontend, libraries like the built-in `fetch` API or third-party libraries like `axios` are used to make HTTP requests, both of which return Promises. `axios` is often preferred for its simpler syntax, automatic JSON transformation, and robust error handling.

Chapter Two: Node.js – Server-Side JavaScript

1. What is Node.js?

Node.js is a runtime environment that allows you to run JavaScript on the server side. It is built on Chrome's V8 JavaScript engine, which provides high performance.

- **Key Features:**

- **Event-Driven Architecture:** Node.js uses an event loop to manage asynchronous operations. Instead of waiting for a task (like reading a file) to complete, it delegates the task and moves on to other code. Once the task is finished, a callback function is placed in the event queue, and the event loop executes it when the main thread is free.
- **Non-Blocking I/O:** Input/Output operations (e.g., reading from a disk, querying a database) are handled asynchronously. This prevents the

main thread from being blocked, allowing Node.js to handle thousands of concurrent connections efficiently.

- **Single-Threaded:** Node.js executes JavaScript code on a single thread. However, it offloads blocking operations (like file I/O) to a background thread pool, which is why it can still be highly scalable.

2. Node.js Core Concepts

- **The Event Loop:** This is the mechanism that enables Node.js to perform non-blocking I/O operations. It continuously checks the event queue for pending tasks (callbacks) and executes them when the main thread is free.
- **Scalability:** Node.js achieves high scalability through its non-blocking I/O and event-driven model. It's ideal for real-time applications (chat apps, gaming servers, streaming platforms) that require high concurrency but not extensive CPU processing. It is not recommended for CPU-intensive tasks (like image processing or complex calculations) because a long-running synchronous task could block the single thread.

3. Setting Up Node.js

- **Installation:** Download and install the LTS (Long-Term Support) version from <https://nodejs.org>.
- **Verification:** After installation, run `node -v` and `npm -v` in your terminal to confirm Node.js and the Node Package Manager (NPM) are installed.

4. Node.js Modules and CommonJS

When Node.js was created, JavaScript lacked a built-in module system. Node.js adopted the **CommonJS** standard to organize and manage code.

- `require()`: Used to import functionality from other modules.
- `module.exports`: Used to export functionality from a module.

```
// math.js
module.exports.add = (a, b) => a + b;

// app.js
const math = require('./math');
console.log(math.add(2, 3)); // 5
```


- **Transition to ES6 Modules:** Modern JavaScript uses `import` and `export`. Node.js supports ES6 modules, but you must either use the file extension `.mjs` or set `"type": "module"` in your `package.json` file.

```
// math.mjs
export const add = (a, b) => a + b;

// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3)); // 5
```

5. Core Node.js Modules

Node.js comes with several built-in modules that provide essential server-side functionality. Some key modules are:

Module	Purpose
<code>fs</code>	File system operations (reading, writing, deleting files and directories).
<code>http</code>	Creating HTTP servers and making HTTP requests.
<code>path</code>	Handling and transforming file paths.
<code>events</code>	Providing an event-driven programming interface.
<code>os</code>	Offering operating system-related utilities.
<code>crypto</code>	Providing cryptographic functionality (hashing, encryption).

6. Building a Basic HTTP Server with the `http` Module

A simple HTTP server can be created using the `http` module.

1. **Create a file** (e.g., `server.js`).
2. **Write the server code:**

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200; // Set HTTP status code
  res.setHeader('Content-Type', 'text/plain'); // Set response header
  res.end('Hello, World!\n'); // Send response body and
```

```

end the response
});

server.listen(3000, 'localhost', () => {
  console.log('Server running at <http://localhost:3000/>');
});

```

- **req (Request Object):** Contains information about the incoming request, such as the method (`req.method`), URL (`req.url`), and headers (`req.headers`).
- **res (Response Object):** Used to send the response back to the client. Key methods include `res.writeHead()` , `res.write()` , and `res.end()` .

3. **Run the server:** In the terminal, run `node server.js` .

7. Working with the File System (`fs`) Module

The `fs` module provides APIs for interacting with the file system. Operations can be performed asynchronously (non-blocking) or synchronously (blocking).

- **Asynchronous (Callback-based):**

```

const fs = require('fs');
fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('File content:', data);
});
console.log('End'); // This will log before the file content

```

- **Asynchronous (Promise-based):**

```

const fs = require('fs').promises;
async function readFile() {
  try {
    const data = await fs.readFile('example.txt', 'utf8');
    console.log(data);
  } catch (err) {

```

```

        console.error(err);
    }
}
readFile();

```

- **Synchronous:**

```

const fs = require('fs');
const data = fs.readFileSync('example.txt', 'utf8');
console.log(data);
console.log('End'); // This will log after the file content

```

8. Understanding Asynchronous Programming Patterns

- **Callbacks:** The original method for handling asynchronicity. A callback is a function passed as an argument that is executed after a task completes. The main drawback is "callback hell," where deeply nested callbacks become difficult to read, debug, and maintain.
- **Promises:** A more manageable way to handle async operations. They represent a future value and can be chained using `.then()` and `.catch()` to avoid nesting.
- **Async/Await:** A modern syntax that makes working with promises even cleaner. It allows you to write asynchronous code that looks and behaves like synchronous code.

9. Managing Projects with NPM (Node Package Manager)

NPM is the default package manager for Node.js, used to install, manage, and share packages.

- **package.json:** This file is the heart of any Node.js project. It contains metadata and a list of all project dependencies. It is created by running `npm init` (or `npm init -y` to use default values).
 - **main:** Specifies the entry point of the application (e.g., `index.js`).
 - **scripts:** Defines custom commands that can be run with `npm run <script-name>`. Common scripts are `start` to run the server and `test` to run tests.

- `dependencies` : Lists packages required for the application to run in production.
- `devDependencies` : Lists packages only needed for development (e.g., testing frameworks, linters, `nodemon`).
- **Key NPM Commands:**
 - `npm init -y` : Initialize a new Node.js project with default values.
 - `npm install <package-name>` : Install a package and add it to `dependencies` .
 - `npm install <package-name> --save-dev` : Install a package as a development dependency.
 - `npm install -g <package-name>` : Install a package globally on your system.
 - `npm run <script-name>` : Execute a script defined in `package.json` .

10. Development Tools: `nodemon`

`nodemon` is a utility that automatically restarts the Node.js server whenever file changes are detected in the project directory. It is typically installed as a development dependency (`npm install -g nodemon` to install globally, or `npm install --save-dev nodemon` for local project use). It can then be used by replacing `node` with `nodemon` in your run command (e.g., `nodemon server.js`).

11. Environment Variables

Environment variables are key-value pairs set outside the application to configure it dynamically. They are crucial for security (keeping sensitive data out of the codebase) and for maintaining flexibility across different environments (development, testing, production).

- **Access:** In Node.js, all environment variables are available in the global `process.env` object. For example, `process.env.PORT` would give you the value of the `PORT` variable.
- **The `.env` File:** For local development, it's common practice to store environment variables in a file named `.env` at the root of your project. This file is listed in `.gitignore` to prevent it from being committed to version control.
- **Loading `.env` Files:** The `dotenv` package is used to load variables from a `.env` file into `process.env` . It must be installed (`npm install dotenv`) and configured at the very top of your application.

```
// CommonJS
require('dotenv').config();
const port = process.env.PORT;

// ES6 Module (with type:module in package.json)
import dotenv from 'dotenv';
dotenv.config();
const port = process.env.PORT;
```